

MODUL PRAKTIKUM #1

10S2002 - Sistem Operasi

MANAJEMEN PROSES LINUX

Minggu	:	5
Setoran	:	• Lembar jawaban • Program
Batas Waktu Setoran	:	Kamis, 4 Maret 2026 pukul 22.00 WIB
Tujuan	:	Mahasiswa mampu menulis program untuk pengelolaan proses pada Linux yang mencakup pembuatan proses dan terminasi proses.

Petunjuk Praktikum:

1. Anda dapat mengerjakan praktikum ini secara berkelompok dengan maksimum 3 mahasiswa per kelompok. Anda bebas memilih anggota kelompok.
2. Setiap kelompok harus menyerahkan setoran sesuai dengan batas waktusetoran. Pada laporan harus dicantumkan identitas anggota kelompok seperti NIM, Nama dan kontribusi dalam mengerjakan tugas ini, seperti contoh

NIM	Nama	Kontribusi
12S24003	Michael Pratama Nasution	Mengerjakan Bagian C
12S24012	Choqy Pananda Sirait	Mengerjakan Bagian A,B
12S24034	Immanuel Alexander Tambunan	Mengerjakan Bagian A,B

3. Anda hanya diperbolehkan diskusi dengan sesama anggota kelompok. Mencontoh pekerjaan dari kelompok lain akan dianggap plagiarisme dan akan ditindak sesuai dengan sanksi akademik yang berlaku di IT Del.
4. Jawaban ditulis dengan tulisan yang rapi di kertas jenis Double Folio bergaris dan jawaban dari setiap soal harus ditulis secara berurutan.
5. Keterlambatan menyerahkan laporan tidak ditolerir dengan alasan apapun. Oleh karena itu, laporan harus dikumpulkan tepat waktu.
6. Gunakan Sistem Operasi Linux boleh menggunakan Distro apapun namun disarankan untuk mempermudah praktikum gunakan Ubuntu.

Referensi:

1. A. Silberschatz, P.B. Galvin, and G. Gagne, Operating System Concepts, 9th edition, Chapter 1 and 2, John Wiley & Sons, Inc., 2013.
2. M. Neil, S. Richard, Beginning Linux Programming, 4th edition, Wiley, 2008.

A. Teori

1. Pertanyaan berikut terkait dengan konsep-konsep dasar proses:
 - a. Jelaskan berkas program atau berkas tereksekusi (*executable file*) serta bagian-bagian memorinya.
 - b. Gambarkan dan jelaskan langkah-langkah umum pemuatan program (*program loading*) sehingga menjadi proses di memori.
 - c. Jelaskan *process control block* (PCB) serta atribut/variabel penting yang dimilikinya.
 - d. Definiskan proses induk (*parent process*) dan proses anak (*child process*).
2. Terdapat 2 sistem yang diterapkan oleh sistem operasi jika ditinjau dari teknik yang digunakannya untuk mengeksekusi sekumpulan atau berbagai jenis program: *batch system* (*jobs system*) dan *time-shared system*. Jelaskan ke-2 sistem tersebut.
3. Misalkan saat ini ada 4 proses di memori yang dieksekusi secara bergantian oleh CPU: process1, process2, process3, dan process4. Gambarkan diagram peralihan konteks (*context switching*) CPU untuk mengeksekusi ke-4 proses tersebut.
4. Berikut ini diberikan kode program sederhana untuk pengelolaan proses di Linux.

```
1 #include <stdio.h>
2 #include <sys/types.h>
3 #include <sys/wait.h>
4 #include <unistd.h>
5
6 int value = 100;
7
8 int main(){
9     pid_t pid;
10
11     pid = fork();
12
13     if(pid < 0) fprintf(stdout,"Fork is failed\n");
14     else if (pid == 0) {
15         value += 10;
16         printf("pid = %d\n", pid);
17         printf("value = %d\n", value);
18         return 0;
19     }
20     else if (pid > 0){
21         wait(NULL);
22         printf("pid = %d\n", pid);
23         printf("value = %d\n",value);
24         return 0;
25     }
26 }
```

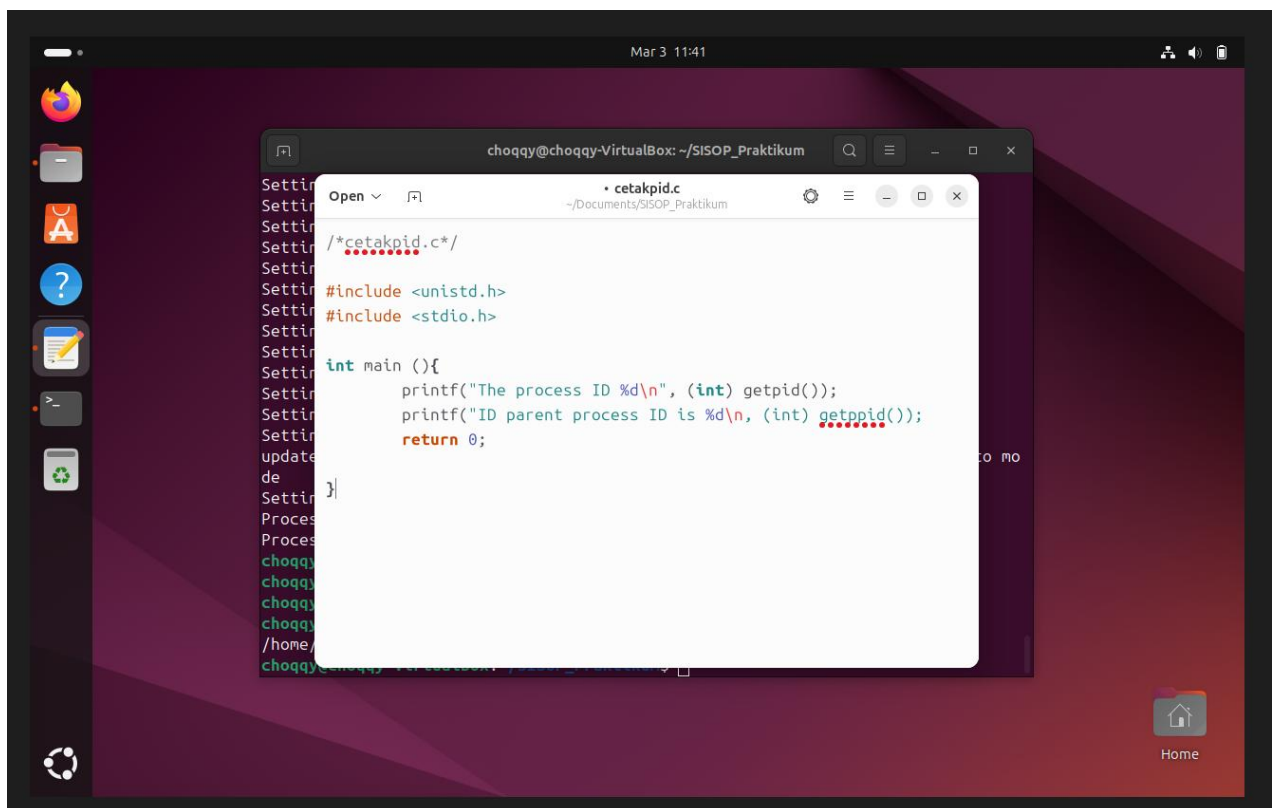
- a. Mengacu ke kode program tersebut, jelaskan tipe data atau fungsi berikut: `pid_t`, `fork()`, dan `wait()`.
- b. Jelaskanlah langkah - langkah pengelolaan proses pada kode di atas dan jelaskan pernyataan - pernyataan mana yang merepresentasikannya.
- c. Mengapa perlu ada 3 blok kondisional setelah `fork` dipanggil ?
- d. Jika proses id induk adalah 2044 dan proses id anak = 2045, apa tampilan pernyataan baris ke-16, ke-17, ke-22, dan ke-23 ?
- e. Kode program tersebut belum menjalankan proses anak, modifikasilah kode program sehingga dengan menjalankan berkas `sh` dengan lokasi: `/bin/sh`. Anda dapat menggunakan fungsi varian fungsi eksekusi proses Linus: `execve`, `execl`, `execlp`.

5. Ada 2 metode dalam melakukan komunikasi antar proses (*inter-process communication – IPC*), yakni *message passing* dan *shared memory*. Jelaskan ke-2 metode tersebut.

B. Pemrograman

1. Tuliskan kode program berikut.

```
1 /*cetakpid.c*/
2
3 #include <unistd.h>
4 #include <stdio.h>
5
6 int main (){
7
8     printf("The process ID %d\n", (int) getpid());
9     printf("ID parent process ID is %d\n", (int) getppid());
10    return 0;
11
12 }
```



Eksekusi program di atas dengan cara sebagai berikut:

```
choqqy@choqqy-VirtualBox:~/SISOP_Praktikum$ gcc cetakpid.c -o cetakpid
choqqy@choqqy-VirtualBox:~/SISOP_Praktikum$ ./cetakpid
The process ID 6338
ID parent process ID is 2623
choqqy@choqqy-VirtualBox:~/SISOP_Praktikum$
```

- a. Jelaskan perbedaan antara **getpid()** dengan **getppid()**.

Jawab : Fungsi **getpid()** digunakan untuk memperoleh Process ID (PID) dari proses yang sedang berjalan. PID adalah nomor unik yang diberikan oleh sistem operasi kepada setiap proses yang dibuat. Dengan kata lain, **getpid()** menunjukkan identitas dari proses itu sendiri.

Sedangkan **getppid()** digunakan untuk memperoleh Parent Process ID (PPID), yaitu nomor identitas dari proses induk yang membuat proses tersebut. Setiap proses di Linux tidak berdiri sendiri, melainkan dibuat oleh proses lain (kecuali proses pertama yang dibuat oleh sistem).

Jadi perbedaannya adalah:

- **getpid()** menampilkan ID proses saat ini.
- **getppid()** menampilkan ID proses induknya.

- b. Jelaskan mengapa setiap kali program di atas dieksekusi, maka akan menampilkan process ID yang berbeda. Jelaskan mengapa?

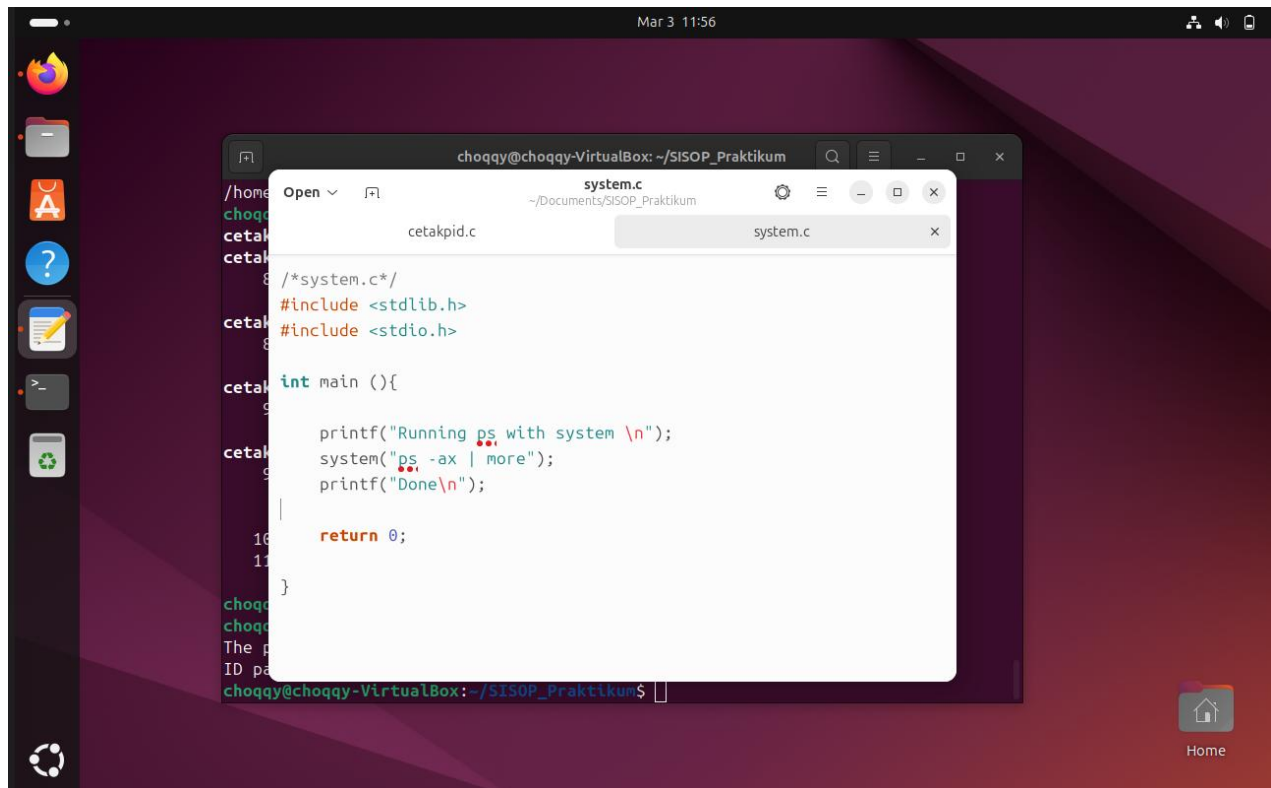
Jawab : setiap kali program dieksekusi, sistem operasi akan membuat sebuah proses baru di memori. Ketika proses baru dibuat, sistem akan memberikan PID yang berbeda dan unik agar dapat dibedakan dari proses lain yang sedang berjalan.

Karena proses sebelumnya sudah selesai (terminated), maka saat program dijalankan kembali, sistem membuat proses baru dengan PID yang berbeda. Sistem operasi tidak menggunakan PID yang sama secara langsung, melainkan mengatur dan mendistribusikan PID sesuai dengan proses yang aktif saat itu.

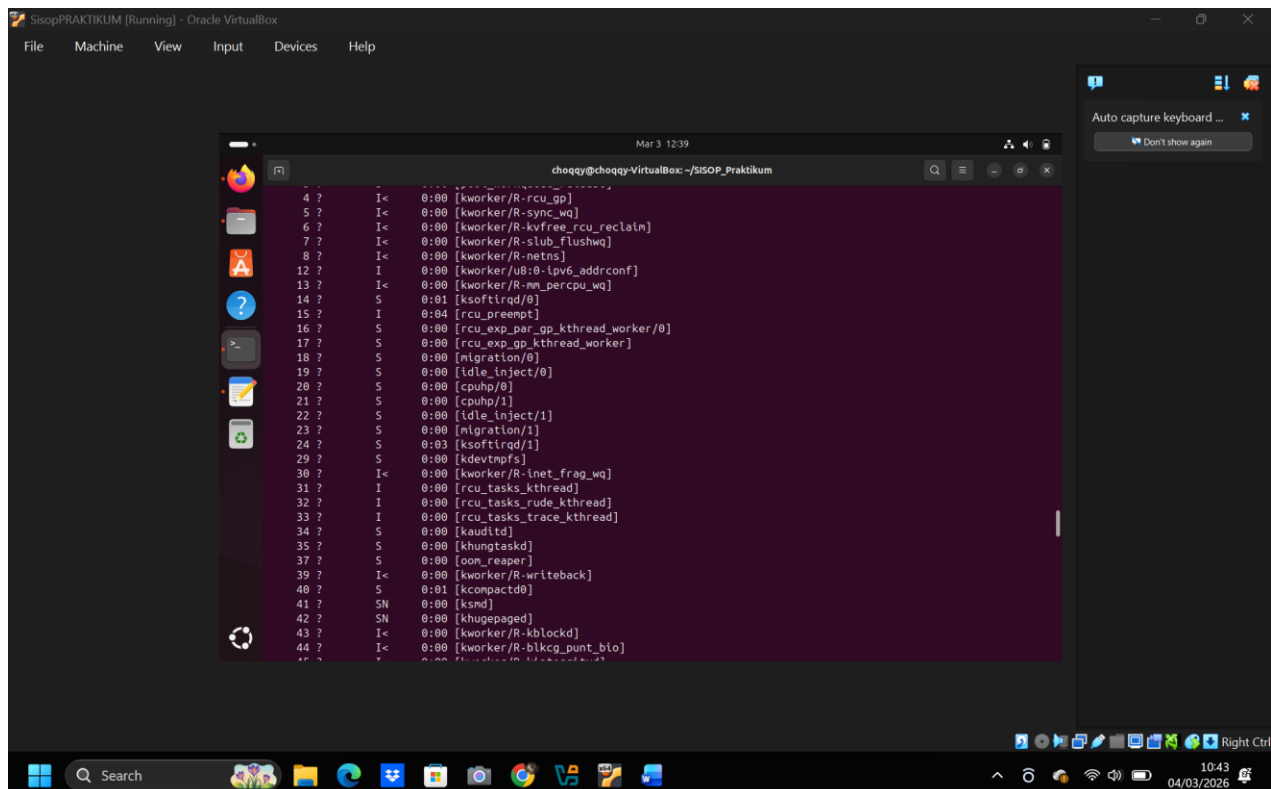
Dengan demikian, setiap eksekusi program menghasilkan PID yang berbeda karena setiap eksekusi dianggap sebagai proses baru oleh sistem operasi.

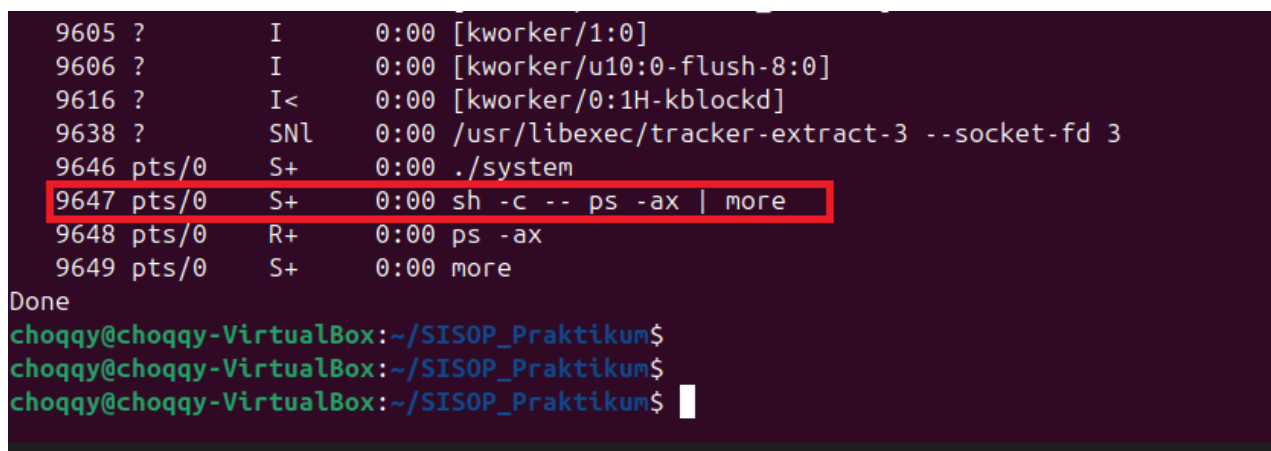
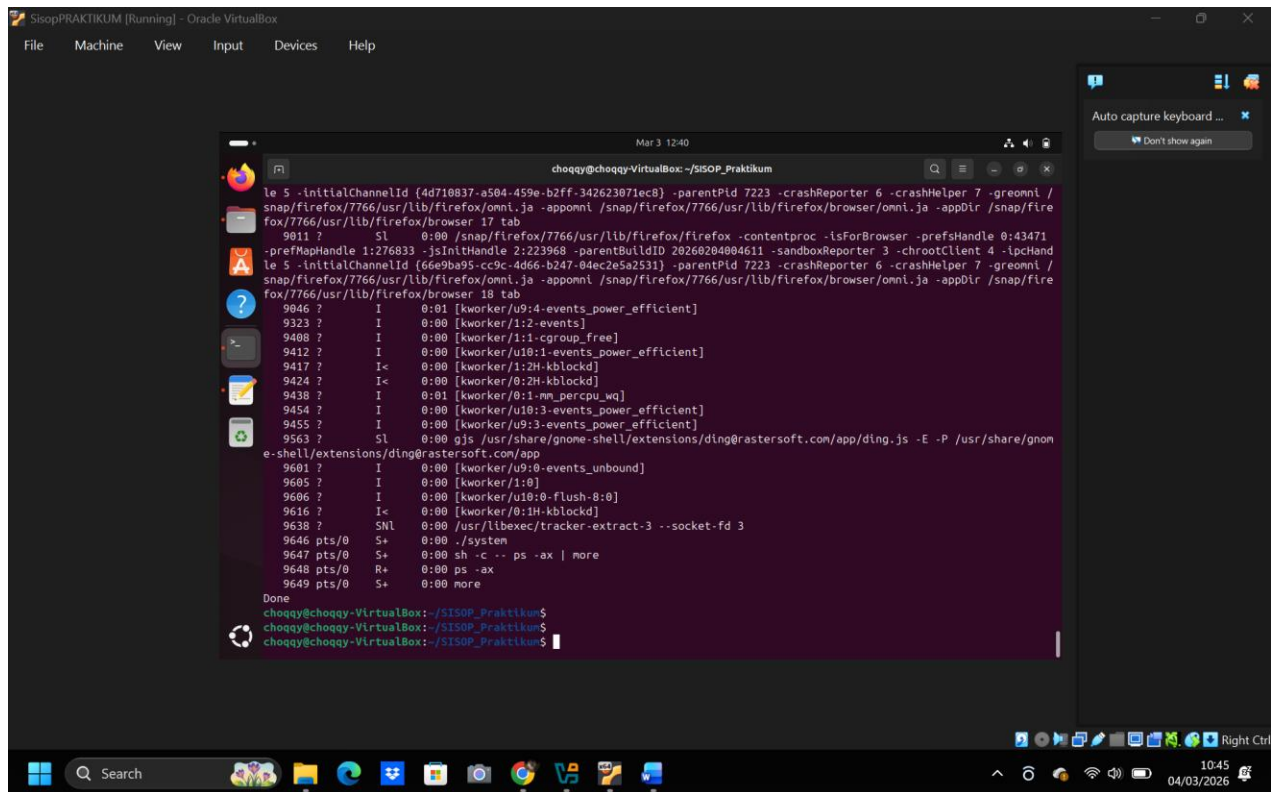
2. Pada kode program berikut, proses baru akan dibentuk dengan menggunakan fungsi **system()**.

```
1 /*system.c*/
2 #include <stdlib.h>
3 #include <stdio.h>
4
5 int main (){
6
7     printf("Running ps with system \n");
8     system("ps -ax | more");
9     printf("Done\n");
10
11     return 0;
12
13 }
```



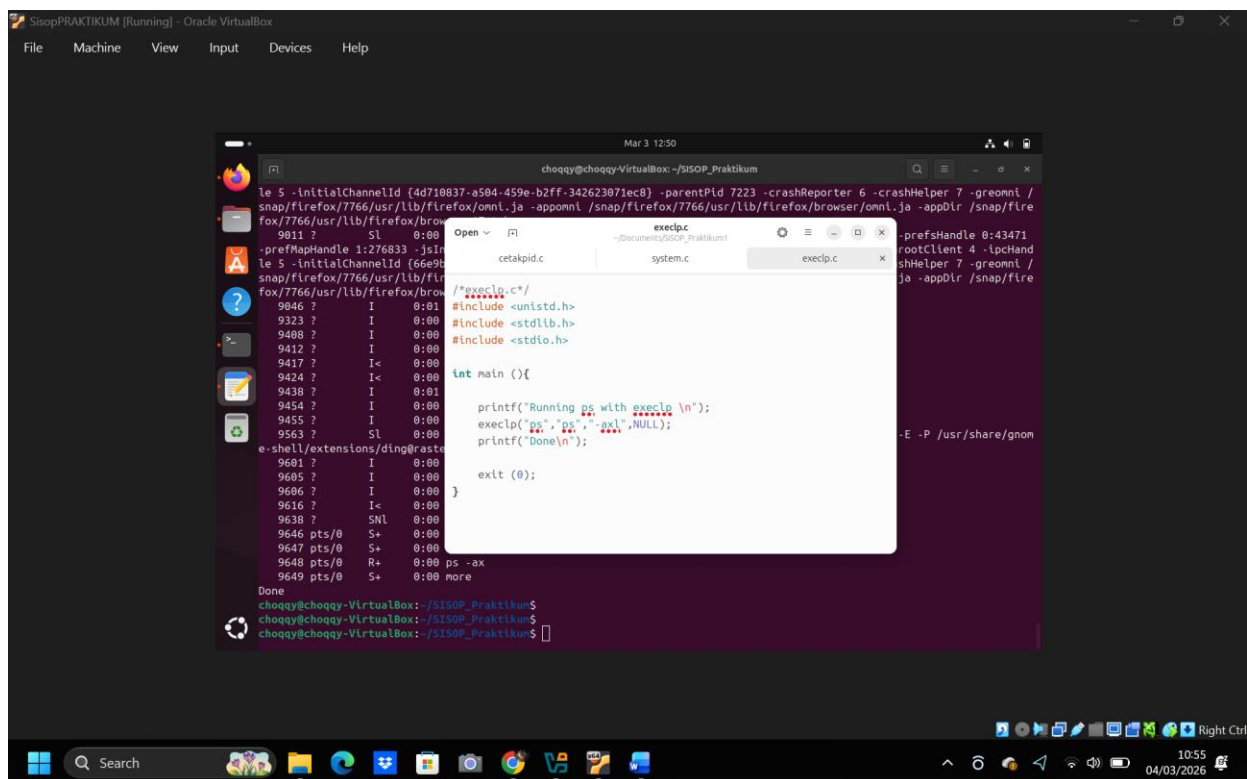
Eksekusilah program di atas kemudian *capture* hasilnya. Tunjukkanlah proses mana yang menjalankan proses **ps -axl | more** dengan menandai ID proses induk-nya.





3. Pada kode program berikut, proses baru akan dibentuk dengan menggunakan fungsi `exec()`.

```
1 /*execlp.c*/
2 #include <unistd.h>
3 #include <stdlib.h>
4 #include <stdio.h>
5
6 int main (){
7
8     printf("Running ps with execlp \n");
9     execlp("ps", "ps", "-axl", NULL);
10    printf("Done\n");
11
12    exit (0);
```



Jalankan kode program pada nomor 3. Amati hasilnya dan bandingkan hasilnya dengan program pada nomor 2. Temukan perbedaannya dan jelaskan mengapa?

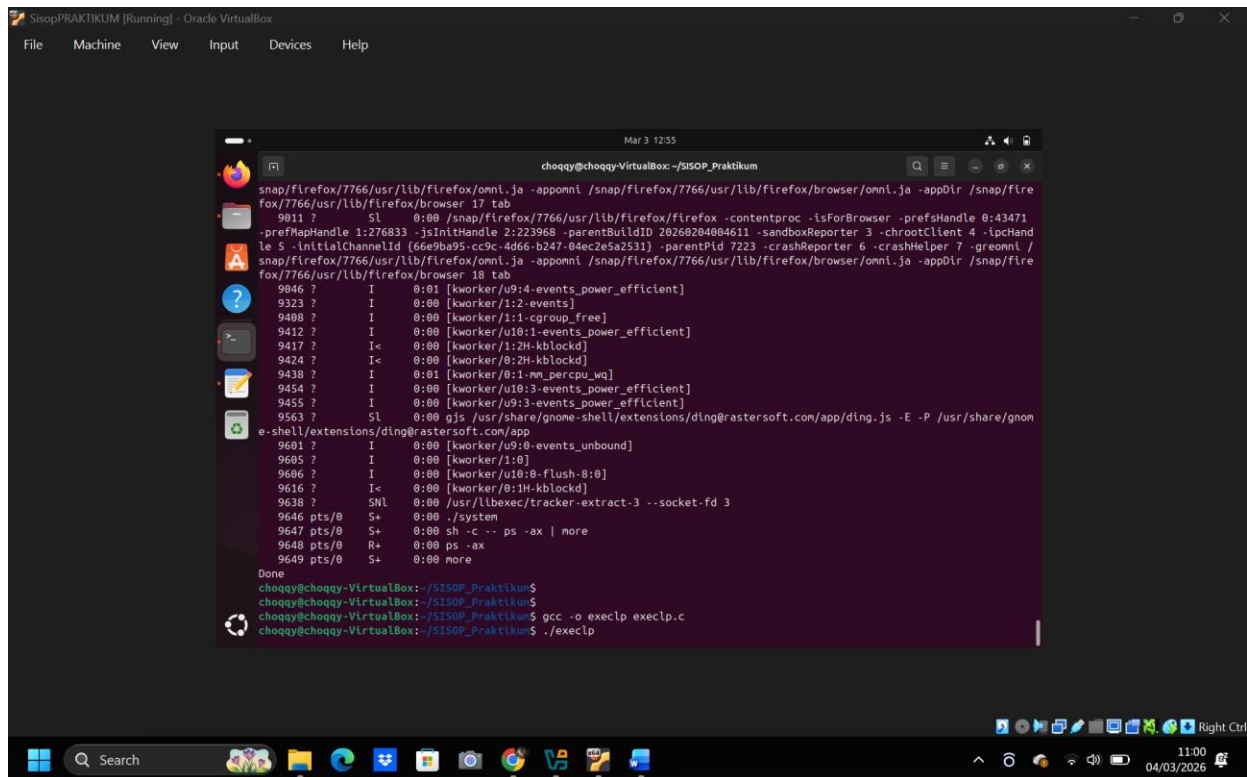
Perbedaan:

Pada program **execlp.c**, fungsi `execlp("ps", "ps", "-axl", NULL);` digunakan untuk menjalankan perintah `ps -axl`. Fungsi `execlp()` akan menggantikan proses yang sedang berjalan dengan proses baru. Artinya, setelah pemanggilan `execlp()`, program asli tidak akan dilanjutkan lagi. Oleh karena itu, perintah seperti `printf("Done\n");` tidak akan dieksekusi, kecuali jika terjadi kesalahan saat memanggil `execlp()`.

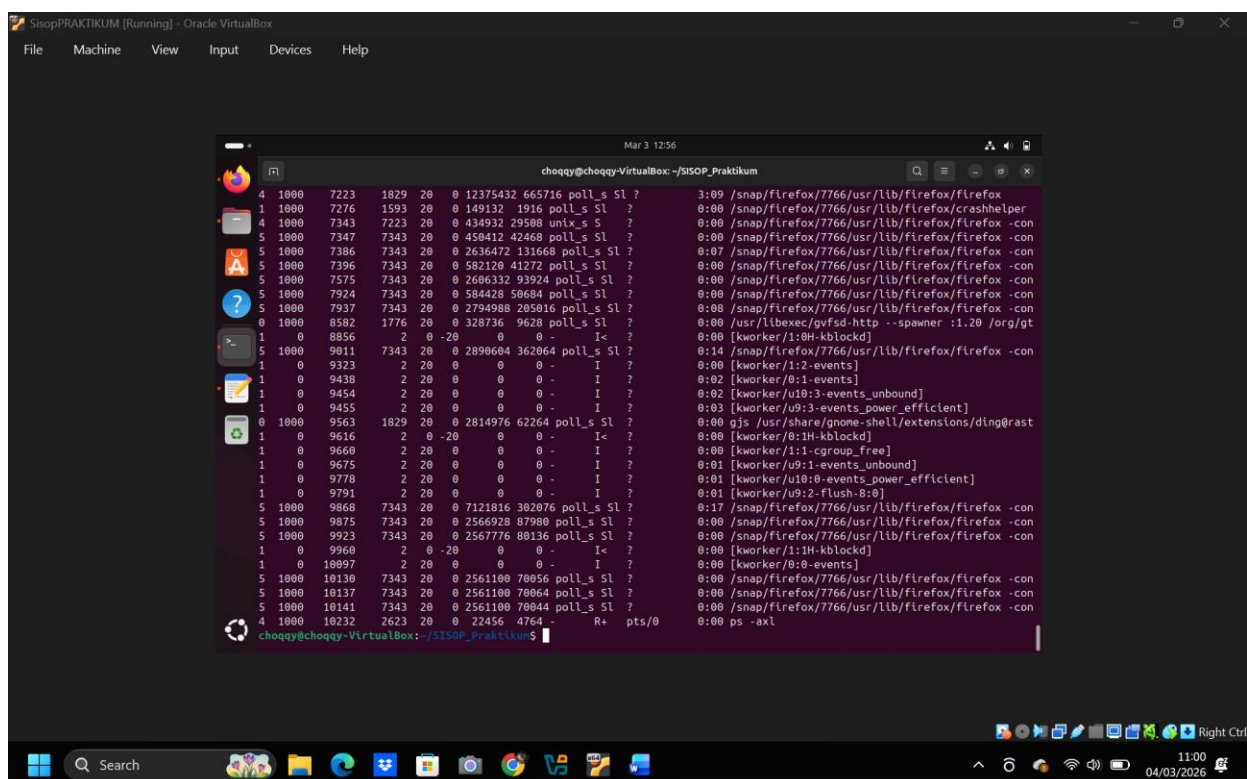
Sedangkan pada program **system.c**, fungsi `system("ps -ax | more");` dipakai untuk mengeksekusi perintah sistem. Berbeda dengan `execlp()`, fungsi `system()` akan membuat proses anak (child process) untuk menjalankan perintah tersebut. Setelah proses anak selesai, kontrol akan kembali ke program utama. Karena itu, perintah

seperti `printf("Done\n");` tetap akan dijalankan setelah eksekusi perintah sistem selesai.

Secara umum, `execvp()` lebih tepat digunakan jika ingin langsung mengganti proses yang sedang berjalan dengan program lain. Sementara itu, `system()` lebih fleksibel karena dapat menjalankan perintah shell yang lebih kompleks, termasuk penggunaan pipe (`|`), tanpa menghentikan proses utama.



```
choqqy@choqqy-VirtualBox: ~/SISOP_Praktikum
snap/firefox/7766/usr/lib/firefox/omni.ja -appomni /snap/firefox/7766/usr/lib/firefox/browser/omni.ja -appdir /snap/firefox/7766/usr/lib/firefox/browser 17 tab
9911 ? I 0:00 /snap/firefox/7766/usr/lib/firefox/firefox-contentproc -lsForBrowser -prefHandle 0:43471
prefMapHandle 1:276833 -jsInitHandle 2:223968 parentBuildID 2026020404611 -sandboxReporter 3 -chromeClient 4 -ipcHand
le 5 -initialChannelId {66e9ba95-cc9c-4d66-b247-04ec2e5a2531} -parentPid 7223 -crashReporter 6 -crashHelper 7 -greonml /
snap/firefox/7766/usr/lib/firefox/omni.ja -appomni /snap/firefox/7766/usr/lib/firefox/browser/omni.ja -appdir /snap/firefox/7766/usr/lib/firefox/browser 18 tab
9946 ? I 0:01 [kworker/u9:3-events_power_efficient]
9323 ? I 0:00 [kworker/1:2-events]
9408 ? I 0:00 [kworker/1:1-cgroup_free]
9412 ? I 0:00 [kworker/u10:1-events_power_efficient]
9417 ? I< 0:00 [kworker/1:2H-kblockd]
9424 ? I< 0:00 [kworker/0:2H-kblockd]
9438 ? I 0:01 [kworker/0:1-mm_percpu_wq]
9454 ? I 0:00 [kworker/u10:3-events_power_efficient]
9455 ? I 0:00 [kworker/u9:3-events_power_efficient]
9563 ? S 0:00 gjs /usr/share/gnome-shell/extensions/ding@rastersoft.com/app/ding.js -E -P /usr/share/gnom
e-shell/extensions/ding@rastersoft.com/app
9601 ? I 0:00 [kworker/u9:0-events_unbound]
9605 ? I 0:00 [kworker/1:0]
9606 ? I 0:00 [kworker/u10:0-flush-8:0]
9616 ? I< 0:00 [kworker/0:1H-kblockd]
9638 ? SNL 0:00 /usr/libexec/tracker-extract-3 --socket-fd 3
9646 pts/0 S+ 0:00 ./system
9647 pts/0 S+ 0:00 sh -c -- ps -ax | more
9648 pts/0 R+ 0:00 ps -ax
9649 pts/0 S+ 0:00 more
Done
choqqy@choqqy-VirtualBox:~/SISOP_Praktikum$
choqqy@choqqy-VirtualBox:~/SISOP_Praktikum$ gcc -o execlp execlp.c
choqqy@choqqy-VirtualBox:~/SISOP_Praktikum$ ./execlp
```



```
choqqy@choqqy-VirtualBox:~/SISOP_Praktikum
4 1000 7223 1829 20 0 12375432 665716 poll_s SL ? 3:09 /snap/firefox/7766/usr/lib/firefox/firefox
1 1000 7276 1593 20 0 149132 1916 poll_s SL ? 0:00 /snap/firefox/7766/usr/lib/firefox/crashhelper
5 1000 7343 7223 20 0 434932 29508 unix_s S ? 0:00 /snap/firefox/7766/usr/lib/firefox/firefox -con
5 1000 7347 7343 20 0 450412 42468 poll_s SL ? 0:00 /snap/firefox/7766/usr/lib/firefox/firefox -con
5 1000 7386 7343 20 0 2636472 131668 poll_s SL ? 0:07 /snap/firefox/7766/usr/lib/firefox/firefox -con
5 1000 7396 7343 20 0 582120 41272 poll_s SL ? 0:00 /snap/firefox/7766/usr/lib/firefox/firefox -con
5 1000 7575 7343 20 0 2606332 93024 poll_s SL ? 0:00 /snap/firefox/7766/usr/lib/firefox/firefox -con
5 1000 7924 7343 20 0 584428 50604 poll_s SL ? 0:00 /snap/firefox/7766/usr/lib/firefox/firefox -con
5 1000 7937 7343 20 0 2794988 285016 poll_s SL ? 0:00 /snap/firefox/7766/usr/lib/firefox/firefox -con
0 1000 8582 1776 20 0 328736 9628 poll_s SL ? 0:00 /usr/libexec/gvfsd-http --spawner :1.20 /org/gt
1 0 8856 2 0 -20 0 0 - I< ? 0:00 [kworker/1:0H-kblockd]
5 1000 9811 7343 20 0 2890604 362064 poll_s SL ? 0:14 /snap/firefox/7766/usr/lib/firefox/firefox -con
1 0 9323 2 0 0 0 - I ? 0:00 [kworker/1:2-events]
1 0 9438 2 0 0 0 - I ? 0:02 [kworker/0:1-events]
1 0 9454 2 0 0 0 - I ? 0:02 [kworker/u10:3-events_unbound]
1 0 9455 2 0 0 0 - I ? 0:03 [kworker/u9:3-events_power_efficient]
0 1000 9563 1829 20 0 2814976 62264 poll_s SL ? 0:00 gjs /usr/share/gnome-shell/extensions/ding@rast
1 0 9616 2 0 -20 0 0 - I< ? 0:00 [kworker/0:1H-kblockd]
1 0 9660 2 0 0 0 - I ? 0:00 [kworker/1:1-cgroup_free]
1 0 9675 2 0 0 0 - I ? 0:01 [kworker/u9:1-events_unbound]
1 0 9778 2 0 0 0 - I ? 0:01 [kworker/u10:0-events_power_efficient]
1 0 9791 2 0 0 0 - I ? 0:01 [kworker/u9:2-flush-8:0]
5 1000 9868 7343 20 0 7121816 302076 poll_s SL ? 0:17 /snap/firefox/7766/usr/lib/firefox/firefox -con
1 1000 9875 7343 20 0 2566928 87908 poll_s SL ? 0:00 /snap/firefox/7766/usr/lib/firefox/firefox -con
5 1000 9923 7343 20 0 2567776 80136 poll_s SL ? 0:00 /snap/firefox/7766/usr/lib/firefox/firefox -con
1 0 9960 2 0 -20 0 0 - I< ? 0:00 [kworker/1:1H-kblockd]
1 0 10097 2 0 0 0 - I ? 0:00 [kworker/0:0-events]
5 1000 10130 7343 20 0 2561100 70056 poll_s SL ? 0:00 /snap/firefox/7766/usr/lib/firefox/firefox -con
5 1000 10137 7343 20 0 2561100 70064 poll_s SL ? 0:00 /snap/firefox/7766/usr/lib/firefox/firefox -con
5 1000 10141 7343 20 0 2561100 70044 poll_s SL ? 0:00 /snap/firefox/7766/usr/lib/firefox/firefox -con
4 1000 10232 2623 20 0 22456 4764 - R+ pts/0 0:00 ps -axl
choqqy@choqqy-VirtualBox:~/SISOP_Praktikum$
```



```

5 1000 10130 7343 20 0 2561100 70056 poll_s SL ? 0:00 /snap/firefox/7766/usr/lib/firefox/firefox -con
5 1000 10137 7343 20 0 2561100 70064 poll_s SL ? 0:00 /snap/firefox/7766/usr/lib/firefox/firefox -con
5 1000 10141 7343 20 0 2561100 70044 poll_s SL ? 0:00 /snap/firefox/7766/usr/lib/firefox/firefox -con
4 1000 10232 2623 20 0 22456 4764 - R+ pts/0 0:00 ps -axl
choqqy@choqqy-VirtualBox:~/SISOP_Praktikum$

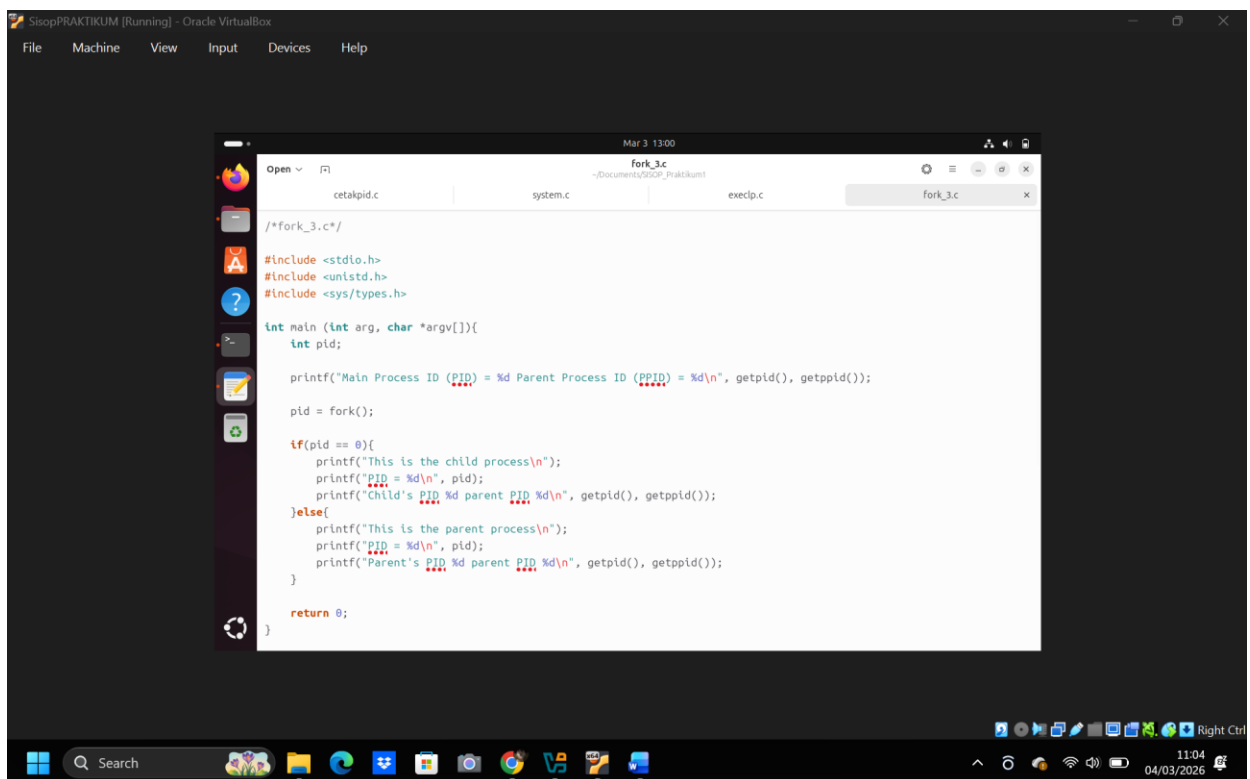
```

4. Tuliskan kode program berikut.

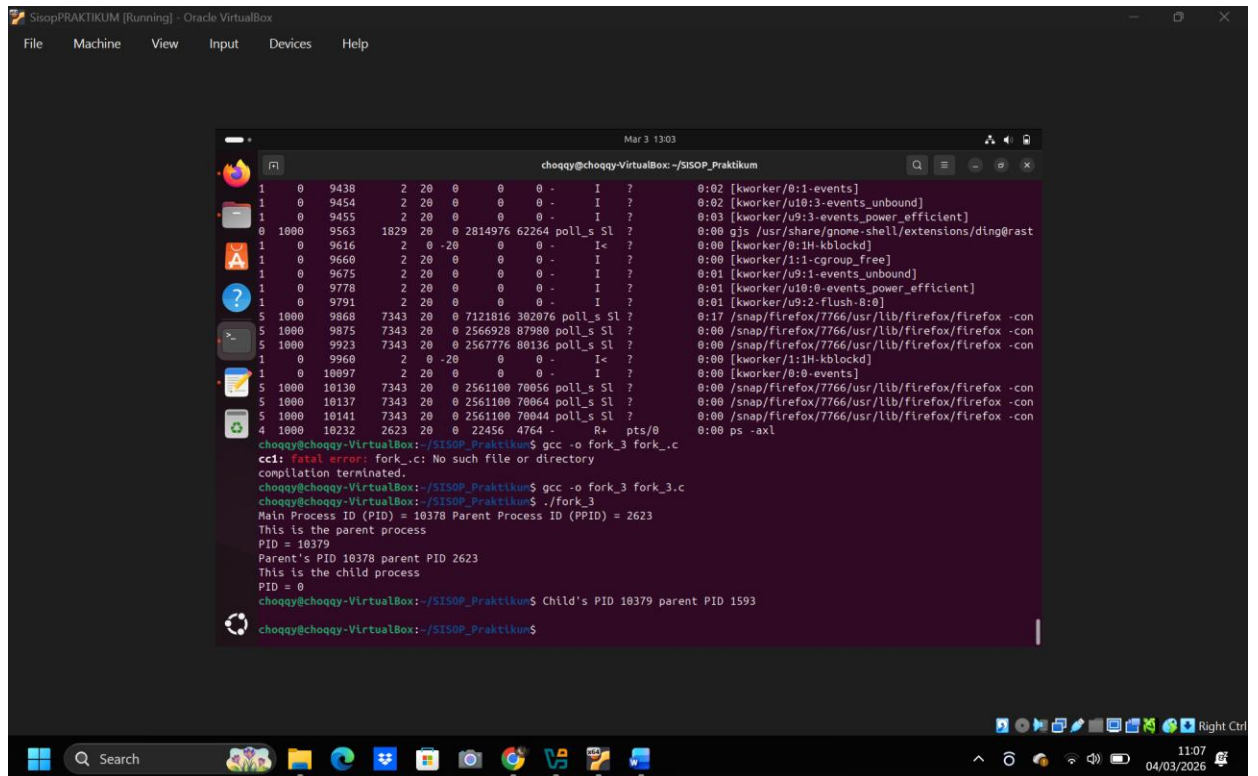
```

1 /*fork_3.c*/
2
3 #include <stdio.h>
4 #include <unistd.h>
5 #include <sys/types.h>
6
7 int main (int arg, char *argv[]){
8     int pid;
9
10    printf("Main Process ID (PID) = %d Parent Process ID (PPID) = %d\n", getpid(), getppid());
11
12    pid = fork();
13
14    if(pid == 0){
15        printf("This is the child process\n");
16        printf("PID = %d\n", pid);
17        printf("Child's PID %d parent PID %d\n", getpid(), getppid());
18    }else{
19        printf("This is the parent process\n");
20        printf("PID = %d\n", pid);
21        printf("Parent's PID %d parent PID %d\n", getpid(), getppid());
22    }
23
24    return 0;
25 }

```



Eksekusi kode program pada nomor 4, amati hasilnya, kemudian jelaskan hasil dari program tersebut. Proses manakah yang dijalankan pertama kali, apakah proses induk atau proses anak? Mengapa?



```
choqgy@choqgy-VirtualBox: ~/SISOP_Praktikum
1 0 9438 2 20 0 0 0 - I ? 0:02 [kworker/0:1-events]
1 0 9454 2 20 0 0 0 - I ? 0:02 [kworker/u10:3-events_unbound]
1 0 9455 2 20 0 0 0 - I ? 0:03 [kworker/u9:3-events_power_efficient]
0 1000 9563 1829 20 0 2814976 62264 poll_s SL ? 0:00 gjs /usr/share/gnome-shell/extensions/ding@rast
1 0 9616 2 0 -20 0 0 - I< ? 0:00 [kworker/0:1H-kblockd]
1 0 9660 2 20 0 0 0 - I ? 0:00 [kworker/1:1-cgroup_free]
1 0 9675 2 20 0 0 0 - I ? 0:01 [kworker/u9:1-events_unbound]
1 0 9778 2 20 0 0 0 - I ? 0:01 [kworker/u10:0-events_power_efficient]
1 0 9791 2 20 0 0 0 - I ? 0:01 [kworker/u9:2-flush-8:0]
5 1000 9868 7343 20 0 7121816 303076 poll_s SL ? 0:17 /snap/firefox/7766/usr/lib/firefox/firefox -con
5 1000 9875 7343 20 0 2566928 87980 poll_s SL ? 0:00 /snap/firefox/7766/usr/lib/firefox/firefox -con
5 1000 9923 7343 20 0 2567776 80136 poll_s SL ? 0:00 /snap/firefox/7766/usr/lib/firefox/firefox -con
1 0 9960 2 0 -20 0 0 - I< ? 0:00 [kworker/1:1H-kblockd]
1 0 10097 2 20 0 0 0 - I ? 0:00 [kworker/0:0-events]
5 1000 10130 7343 20 0 2561100 70056 poll_s SL ? 0:00 /snap/firefox/7766/usr/lib/firefox/firefox -con
5 1000 10137 7343 20 0 2561100 70064 poll_s SL ? 0:00 /snap/firefox/7766/usr/lib/firefox/firefox -con
5 1000 10141 7343 20 0 2561100 70044 poll_s SL ? 0:00 /snap/firefox/7766/usr/lib/firefox/firefox -con
4 1000 10232 2623 20 0 22456 4764 - R+ pts/0 0:00 ps -axl
choqgy@choqgy-VirtualBox:~/SISOP_Praktikum$ gcc -o fork_3 fork_3.c
cc1: fatal error: fork_3.c: No such file or directory
compilation terminated.
choqgy@choqgy-VirtualBox:~/SISOP_Praktikum$ gcc -o fork_3 fork_3.c
choqgy@choqgy-VirtualBox:~/SISOP_Praktikum$ ./fork_3
Main Process ID (PID) = 10378 Parent Process ID (PPID) = 2623
This is the parent process
PID = 10379
Parent's PID 10378 parent PID 2623
This is the child process
PID = 0
choqgy@choqgy-VirtualBox:~/SISOP_Praktikum$ Child's PID 10379 parent PID 1593
choqgy@choqgy-VirtualBox:~/SISOP_Praktikum$
```

1. Penjelasan Hasil Eksekusi Program

Berdasarkan log di terminal, urutan keluaran yang muncul adalah:

Identitas Awal: Program mencetak Main Process ID (PID) = 10378 dengan Parent Process ID (PPID) = 2623.

Blok Induk (Parent): Kalimat "This is the parent process" muncul lebih dulu. Induk melaporkan PID anaknya adalah 10379.

Blok Anak (Child): Kalimat "This is the child process" muncul kemudian. Di dalam proses anak, variabel pid bernilai 0.

2. Proses yang Dijalankan Pertama Kali yaitu Proses induk (parent)

Pada eksekusi tersebut, proses induk (parent) dijalankan pertama kali setelah pemanggilan fork().

3. Alasannya adalah sebagai berikut :

Hal ini terjadi karena beberapa alasan teknis dalam sistem operasi:

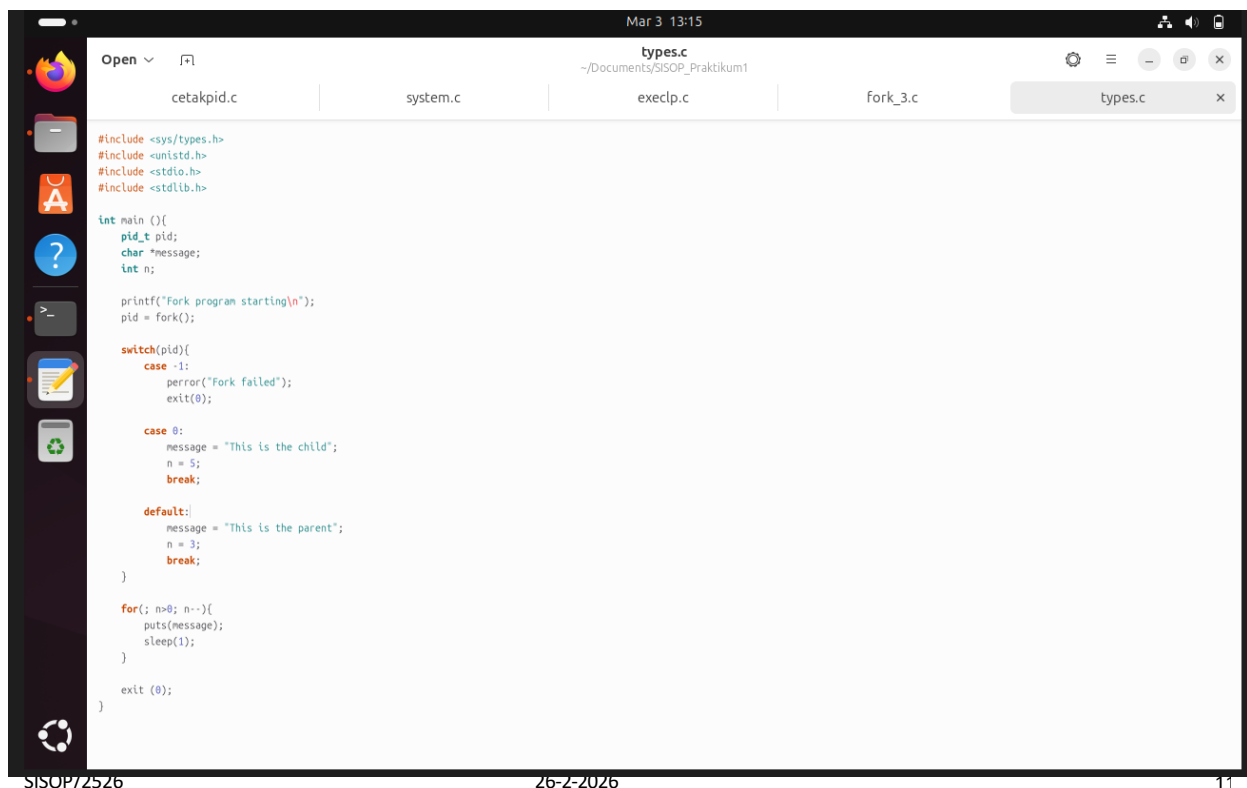
Penjadwalan CPU (CPU Scheduling): Setelah fungsi fork() dipanggil, sistem operasi memiliki dua proses yang sama-sama siap dijalankan (*ready state*). Keputusan tentang siapa yang mendapatkan giliran menggunakan CPU pertama kali sepenuhnya diatur oleh *scheduler* kernel Linux.

Nondeterministik: Urutan eksekusi ini bersifat acak. Pada eksekusi lain, ada kemungkinan proses anak berjalan lebih dulu. Hal ini sangat bergantung pada beban sistem dan algoritma penjadwalan saat program dijalankan.

Ketidakhadiran Sinkronisasi: Kode program tersebut tidak menggunakan fungsi wait(), sehingga proses induk tidak menunggu proses anak selesai. Akibatnya, kedua proses saling "berlomba" (*race condition*) untuk mencetak teks ke layar.

5. Tuliskan kode program berikut.

```
1 #include <sys/types.h>
2 #include <unistd.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5
6 int main (){
7     pid_t pid;
8     char *message;
9     int n;
10
11     printf("Fork program starting\n");
12     pid = fork();
13
14     switch(pid){
15         case -1:
16             perror("Fork failed");
17             exit(0);
18
19         case 0:
20             message = "This is the child";
21             n = 5;
22             break;
23
24         default:
25             message = "This is the parent";
26             n = 3;
27             break;
28     }
29
30     for(; n>0; n--){
31         puts(message);
32         sleep(1);
33     }
34
35     exit (0);
```



Program di atas akan menjalankan dua proses yaitu proses induk dan proses anak. Proses anak akan dijalankan sebanyak 5 kali, dan proses induk akan dijalankan sebanyak 3 kali. Jalankan program di atas dan amati hasil yang terjadi.

```

choqgy@choqgy-VirtualBox: ~/SISOP_Praktikum
Main Process ID (PID) = 18378 Parent Process ID (PPID) = 2623
This is the parent process
PID = 18379
Parent's PID 18378 parent PID 2623
This is the child process
PID = 0
choqgy@choqgy-VirtualBox:~/SISOP_Praktikum$ gcc -o 5 types.c
choqgy@choqgy-VirtualBox:~/SISOP_Praktikum$ ./5
Fork program starting
This is the parent
This is the child
This is the parent
This is the child
This is the parent
This is the child
choqgy@choqgy-VirtualBox:~/SISOP_Praktikum$ This is the child
choqgy@choqgy-VirtualBox:~/SISOP_Praktikum$ ./5
Fork program starting
This is the parent
This is the child
This is the parent
This is the child
This is the parent
This is the child
choqgy@choqgy-VirtualBox:~/SISOP_Praktikum$ This is the child
choqgy@choqgy-VirtualBox:~/SISOP_Praktikum$ ./5
Fork program starting
This is the parent
This is the child
This is the parent
This is the child
This is the parent
This is the child
choqgy@choqgy-VirtualBox:~/SISOP_Praktikum$ This is the child
choqgy@choqgy-VirtualBox:~/SISOP_Praktikum$ ./5
Fork program starting
This is the parent
This is the child
This is the parent
This is the child
This is the parent
This is the child
choqgy@choqgy-VirtualBox:~/SISOP_Praktikum$ This is the child
choqgy@choqgy-VirtualBox:~/SISOP_Praktikum$
  
```

Program ini menggunakan `fork()` untuk membuat proses anak (*child process*) dari proses induk (*parent process*). Setelah pemanggilan `fork()`, terdapat dua proses yang berjalan secara paralel:

1. Proses induk (*parent*) menjalankan pesan sebanyak 3 kali.
2. Proses anak (*child*) menjalankan pesan sebanyak 5 kali.

Penjelasan Output:

Baris pertama (Fork program starting) ditampilkan oleh *parent* sebelum `fork()` dipanggil. *Parent* dan *child* berjalan secara bersamaan setelah `fork()`, mencetak pesan masing-masing secara bergantian.

Parent mencetak "This is the parent" sebanyak 3 kali sebelum selesai.

Child mencetak "This is the child" sebanyak 5 kali sebelum selesai.

Proses yang Dijalankan Pertama Kali:

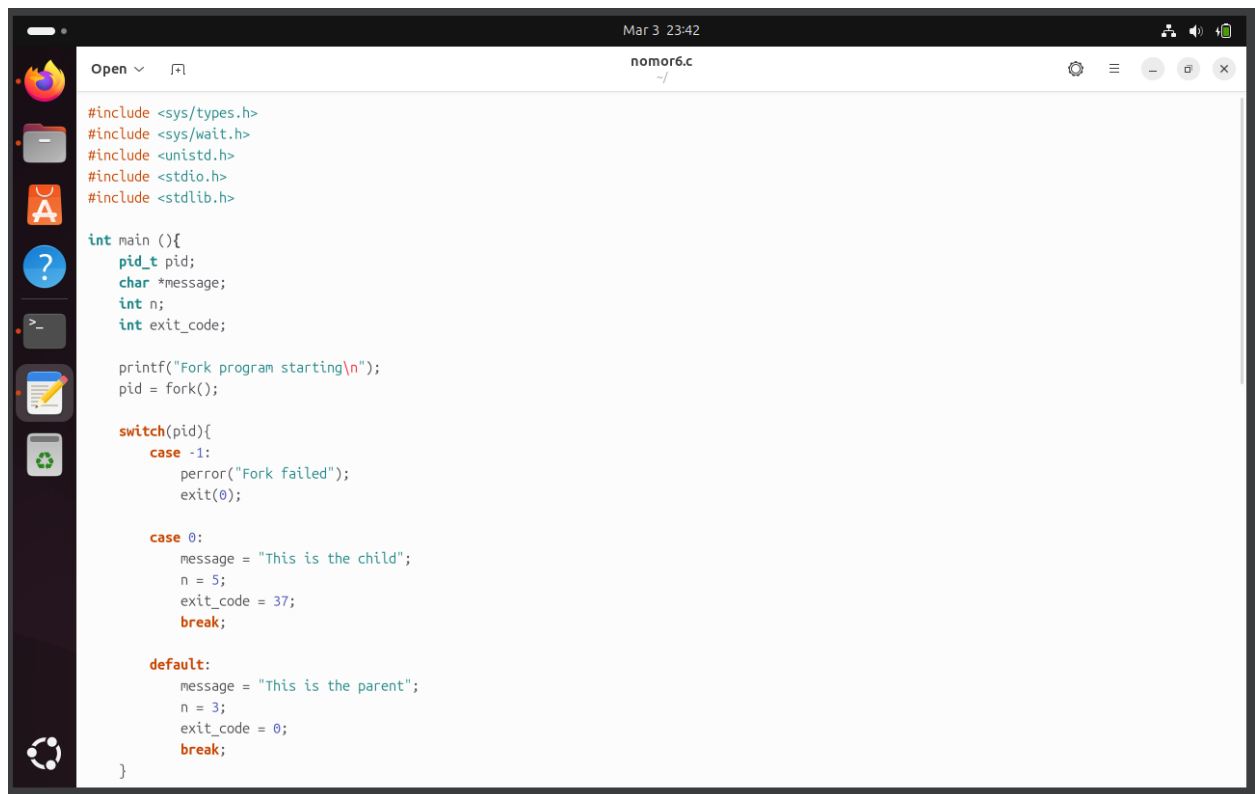
Berdasarkan gambar hasil eksekusi, proses induk (*parent*) dijalankan pertama kali setelah pemanggilan `fork()`.

Hal ini terjadi karena:

- Proses *parent* dan *child* berjalan secara independen.
- Sistem operasi menentukan proses mana yang dieksekusi terlebih dahulu melalui mekanisme *scheduler*.
- Penjadwalan CPU bisa menyebabkan variasi dalam urutan cetakan, dan pada eksekusi ini, *parent* mendapatkan giliran lebih dulu.
- Pada gambar hasil eksekusi, setelah *shell prompt* (choqgy@choqgy-VirtualBox), masih ada pesan "This is the child" yang dicetak. Ini terjadi karena proses *parent* selesai lebih cepat (hanya 3 iterasi), sehingga *shell* mendapatkan kontrol kembali, sementara proses anak masih berjalan (5 iterasi) dan sisa pesannya tetap dicetak setelah *prompt* muncul.

6. Sesuai dengan siklus pembentukan proses anak dengan menggunakan fork() hingga proses anak diterminasi seperti yang ditunjukkan pada slide presentasi halaman 3.21, maka proses induk harus menunggu seluruh proses anak selesai dengan memanggil system call wait(). Dengan memodifikasi program nomor 5, tuliskan program di bawah (halaman selanjutnya). Pada program tersebut anda akan menerapkan system call wait() dengan menggunakan library sys/wait.h.

```
1 #include <sys/types.h>
2 #include <sys/wait.h>
3 #include <unistd.h>
4 #include <stdio.h>
5 #include <stdlib.h>
6
7 int main (){
8     pid_t pid;
9     char *message;
10    int n;
11    int exit_code;
12
13    printf("Fork program starting\n");
14    pid = fork();
15
16    switch(pid){
17        case -1:
18            perror("Fork failed");
19            exit(0);
20
21        case 0:
22            message = "This is the child";
23            n = 5;
24            exit_code = 37;
25            break;
26
27        default:
28            message = "This is the parent";
29            n = 3;
30            exit_code = 0;
31            break;
32    }
33
34    for(; n>0; n--){
35        puts(message);
36        sleep(1);
37    }
38
39    if(pid!=0){
40        int stat_val;
41        pid_t child_pid;
42
43        child_pid = wait(&stat_val);
44
45        printf("Child has finished with PID = %d\n", child_pid);
46        if(WIFEXITED(stat_val)){
47            printf("Child exited with code %d\n", WEXITSTATUS(stat_val));
48        }else{
49            printf("Child terminated abnormally\n");
50        }
51    }
52
53    exit(exit_code);
```

```
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>

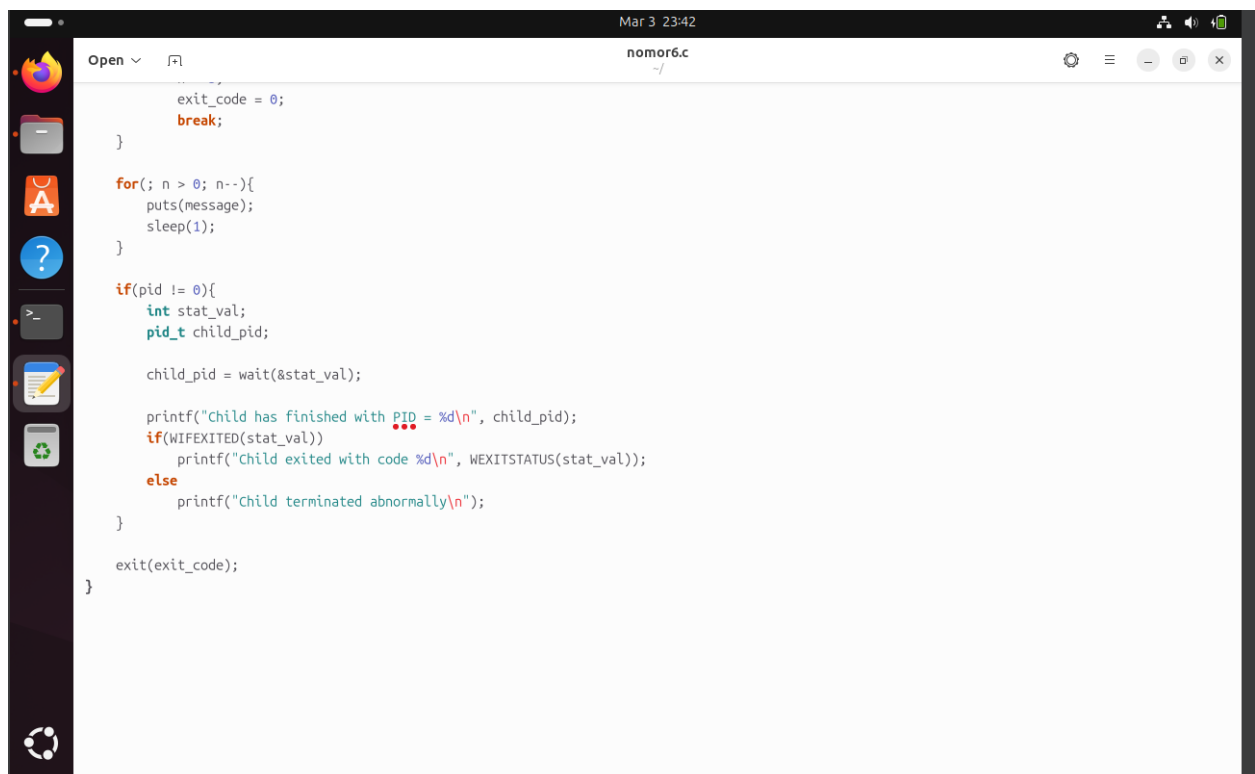
int main (){
    pid_t pid;
    char *message;
    int n;
    int exit_code;

    printf("Fork program starting\n");
    pid = fork();

    switch(pid){
        case -1:
            perror("Fork failed");
            exit(0);

        case 0:
            message = "This is the child";
            n = 5;
            exit_code = 37;
            break;

        default:
            message = "This is the parent";
            n = 3;
            exit_code = 0;
            break;
    }
}
```



```
        exit_code = 0;
        break;
    }

    for(; n > 0; n--){
        puts(message);
        sleep(1);
    }

    if(pid != 0){
        int stat_val;
        pid_t child_pid;

        child_pid = wait(&stat_val);

        printf("Child has finished with PID = %d\n", child_pid);
        if(WIFEXITED(stat_val))
            printf("Child exited with code %d\n", WEXITSTATUS(stat_val));
        else
            printf("Child terminated abnormally\n");
    }

    exit(exit_code);
}
```

```
Mar 3 23:43
immanuel@immanuel-VirtualBox: ~
Setting up libstdc++-13-dev:amd64 (13.3.0-6ubuntu2-24.04.1) ...
Setting up libalgorithm-merge-perl (0.08-5) ...
Setting up g++-13-x86-64-linux-gnu (13.3.0-6ubuntu2-24.04.1) ...
Setting up dpkg-dev (1.22.6ubuntu6.5) ...
Setting up g++-x86-64-linux-gnu (4:13.2.0-7ubuntu1) ...
Setting up g++-13 (13.3.0-6ubuntu2-24.04.1) ...
Setting up g++ (4:13.2.0-7ubuntu1) ...
update-alternatives: using /usr/bin/g++ to provide /usr/bin/c++ (c++) in auto mo
de
Setting up build-essential (12.10ubuntu1) ...
Processing triggers for man-db (2.12.0-4build2) ...
Processing triggers for libc-bin (2.39-0ubuntu8.7) ...
immanuel@immanuel-VirtualBox:~$ gcc --version
gcc (Ubuntu 13.3.0-6ubuntu2-24.04.1) 13.3.0
Copyright (C) 2023 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

immanuel@immanuel-VirtualBox:~$ gcc nomor6.c -o 6
immanuel@immanuel-VirtualBox:~$ ./6
Fork program starting
This is the child
This is the parent
This is the child
This is the parent
This is the child
This is the parent
This is the child
This is the child
Child has finished with PID = 6526
Child exited with code 37
immanuel@immanuel-VirtualBox:~$
```

```
Mar 3 23:45
immanuel@immanuel-VirtualBox: ~
immanuel@immanuel-VirtualBox:~$ gcc --version
gcc (Ubuntu 13.3.0-6ubuntu2-24.04.1) 13.3.0
Copyright (C) 2023 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

immanuel@immanuel-VirtualBox:~$ gcc nomor6.c -o 6
immanuel@immanuel-VirtualBox:~$ ./6
Fork program starting
This is the child
This is the parent
This is the child
This is the parent
This is the child
This is the parent
This is the child
This is the child
Child has finished with PID = 6526
Child exited with code 37
immanuel@immanuel-VirtualBox:~$ ./6
Fork program starting
This is the parent
This is the child
This is the parent
This is the child
This is the parent
This is the child
This is the child
Child has finished with PID = 6658
Child exited with code 37
immanuel@immanuel-VirtualBox:~$
```

7. Jelaskan apa efek dari menggunakan fungsi wait() dari program di atas? Bandingkan hasil dari program pada nomor 5 dengan hasil program nomor 6, apa yang dapat anda simpulkan dari kedua program tersebut?

Efek Menggunakan Fungsi wait()

Berdasarkan kedua program yaitu nomor 5 dengan 6, dapat disimpulkan bahwa:

1. Program Nomor 5 menunjukkan perilaku asinkron di mana induk tidak peduli pada status anak, yang dapat menyebabkan munculnya sisa output setelah program utama dianggap selesai oleh sistem.

2. Program Nomor 6 menunjukkan manajemen proses yang lebih baik melalui sinkronisasi, memastikan seluruh tugas anak terpantau oleh induk sebelum seluruh rangkaian program benar-benar berhenti. Penggunaan `wait()` sangat penting untuk menghindari adanya proses yang menggantung atau tidak terpantau oleh sistem.

Jadi, Fungsi `wait(&stat_val)` pada program berfungsi untuk mensinkronisasi proses induk dan proses anak. Efek utamanya adalah menunda eksekusi proses induk (*parent*) dan memaksanya untuk menunggu hingga proses anak (*child*) selesai dijalankan sepenuhnya. Setelah anak selesai, induk akan mengambil status keluar (*exit status*) anak tersebut, lalu melanjutkan sisa instruksinya untuk mencetak PID anak dan kode status keluar.

8. Jelaskan mengapa parent process harus memanggil system call `wait()` dan apa yang terjadi apabila system call `wait()` tidak dipanggil?

Dalam arsitektur Unix/Linux, interaksi antara proses induk (*parent*) dan proses anak (*child*) diatur secara ketat. Pemanggilan `wait()` bukan sekadar formalitas, melainkan mekanisme kontrol utama.

Mengapa Parent Process Wajib Memanggil `wait()`?

1. Pembersihan Tabel Proses (Mencegah *Zombie*): Saat sebuah proses anak selesai, sistem operasi tidak langsung menghapus seluruh datanya. Sistem menyimpan informasi kecil (seperti status keluar) di tabel proses. `wait()` berfungsi sebagai instruksi bagi induk untuk "menjemput" informasi tersebut sehingga slot di tabel proses bisa benar-benar dibebaskan.
2. Sinkronisasi Operasi: `wait()` bertindak sebagai titik koordinasi. Induk sering kali membutuhkan hasil kerja anak sebelum melanjutkan langkah berikutnya. Dengan `wait()`, induk dipaksa berhenti sejenak hingga anak menyelesaikan tugasnya, menjaga alur program tetap logis dan runtut.
3. Manajemen Status Keluar: Hanya melalui `wait()`, induk dapat mengetahui apakah anak selesai dengan sukses (kode 0) atau mengalami eror. Tanpa ini, induk akan "buta" terhadap keberhasilan tugas yang ia delegasikan.

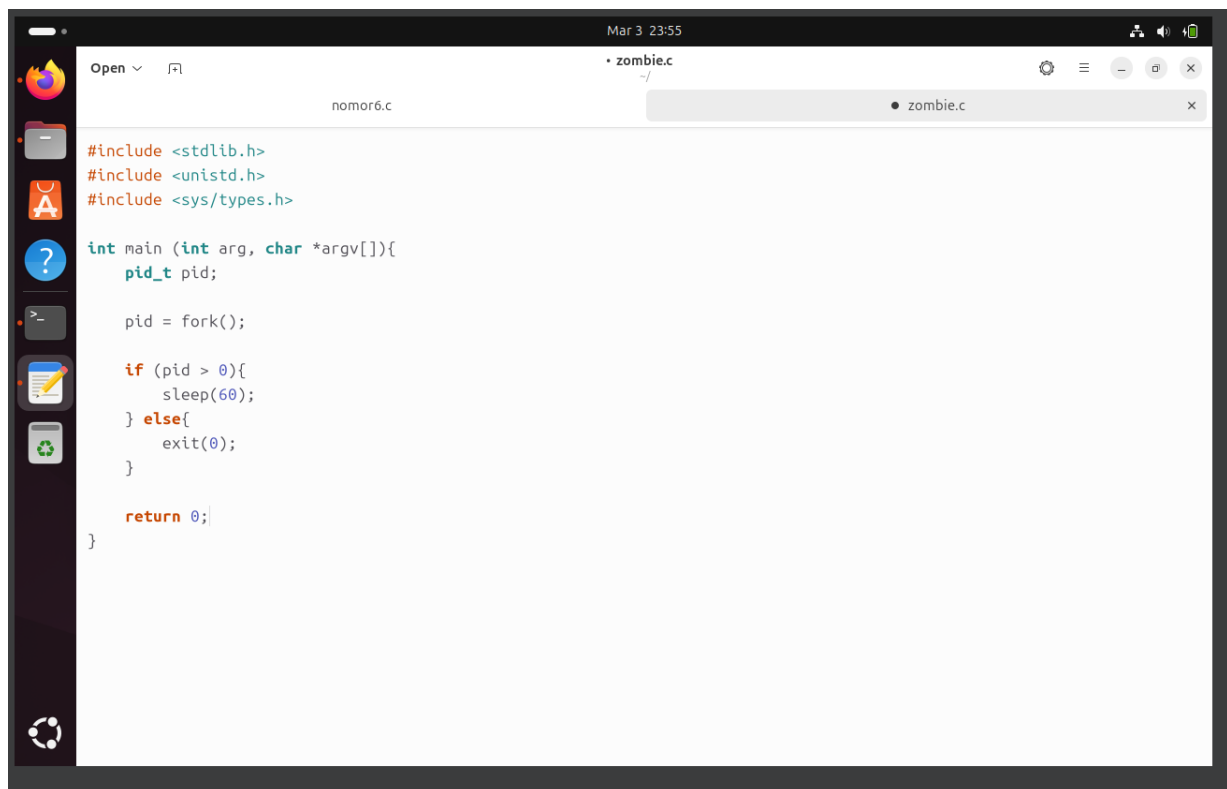
Konsekuensi Jika `wait()` Diabaikan

Jika seorang pengembang lupa atau sengaja tidak menyertakan `wait()`, akan muncul beberapa masalah sistemik:

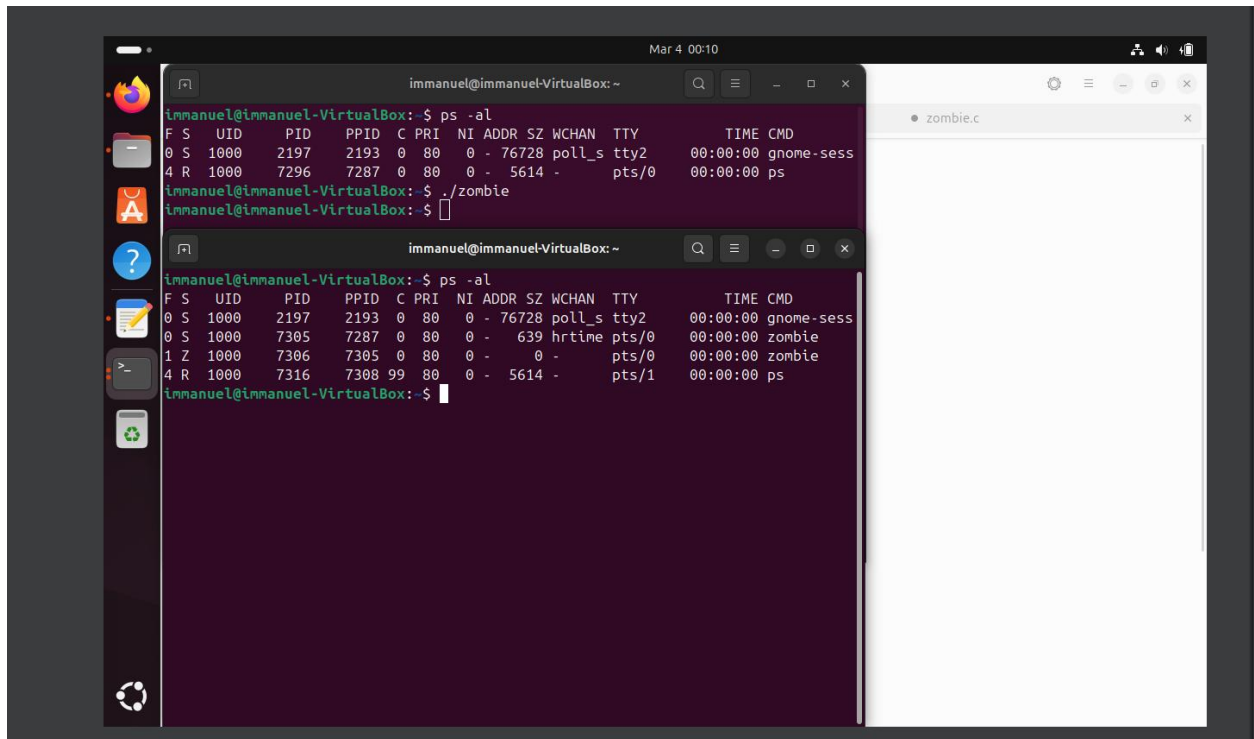
- Akumulasi *Zombie Process*: Proses anak yang telah mati namun statusnya tidak diambil oleh induk akan tetap berada di tabel proses sebagai "Zombie". Jika jumlah zombie membengkak, tabel proses akan penuh (mencapai batas `PID_MAX`), yang mengakibatkan sistem tidak bisa menjalankan program baru sama sekali.
- Terbentuknya *Orphan Process* (Proses Yatim): Jika induk selesai lebih dulu tanpa mempedulikan anaknya, si anak akan menjadi "yatim". Meski secara otomatis akan diadopsi oleh proses `init` (atau `systemd`), hal ini sering kali menandakan desain program yang kurang rapi dan sulit dipantau perilakunya.
- Kekacauan Data (*Race Condition*): Induk mungkin mencoba mengakses file atau memori yang seharusnya diproses oleh anak terlebih dahulu. Tanpa menunggu via `wait()`, induk bisa saja mengolah data yang belum siap atau masih kosong, yang berujung pada *crash* atau *output* yang salah.

9. Tuliskan kode program berikut.

```
1 /*zombie.c*/
2
3 #include <stdlib.h>
4 #include <unistd.h>
5 #include <sys/types.h>
6
7 int main (int arg, char *argv[]){
8     pid_t pid;
9
10    pid = fork();
11
12    if (pid > 0){
13        sleep(60);
14    }else{
15        exit (0);
16    }
17
18    return 0;
19 }
```



10. Eksekusi program di atas. Observasi hasil dari eksekusi program, Anda dapat menggunakan perintah `ps -al` pada terminal yang lain untuk melihat proses yang sedang berjalan. Apakah hasil dari perintah `ps -al`, jelaskan mengapa terjadi hal demikian?



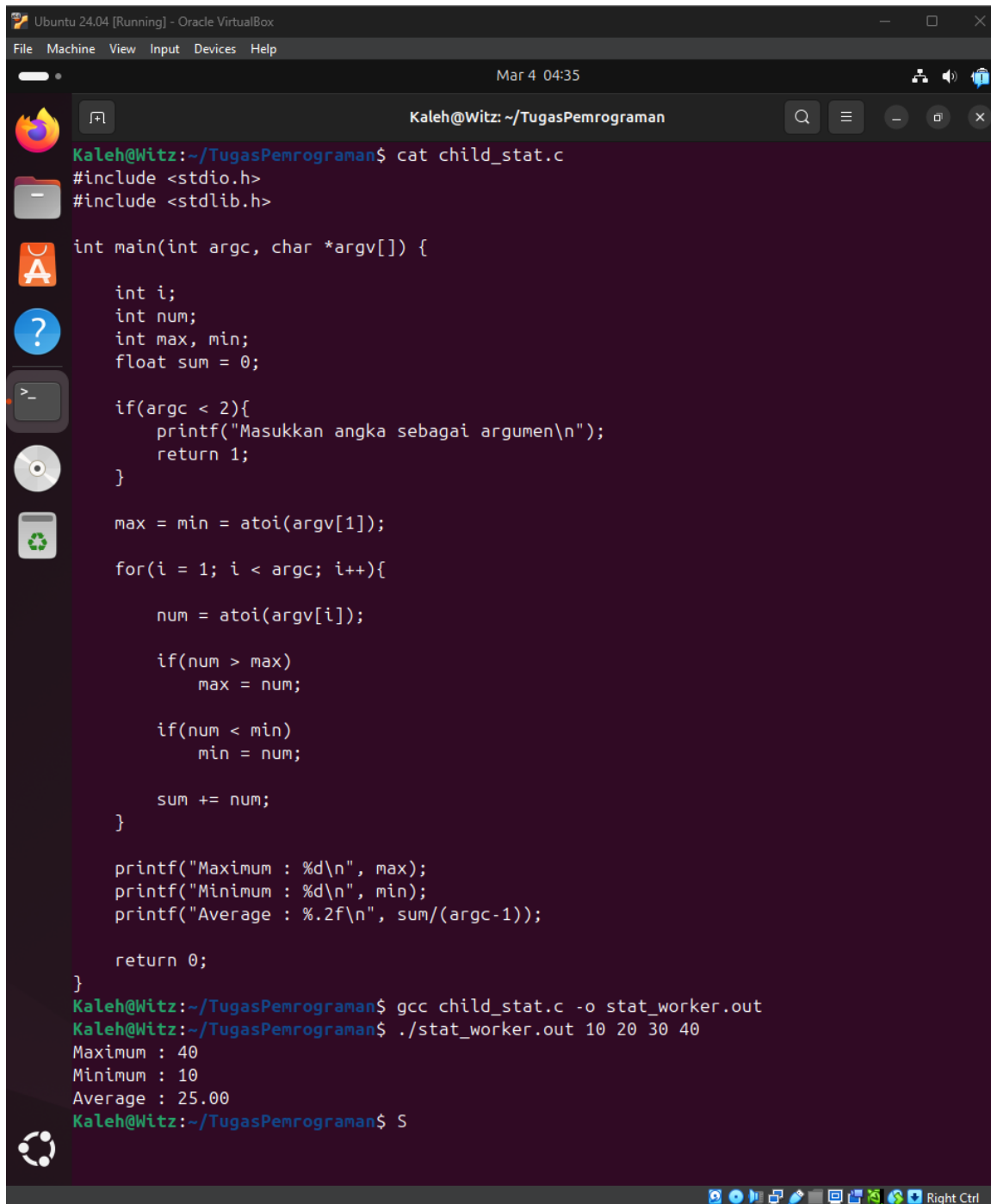
Berdasarkan hasil pengamatan menggunakan perintah `ps -al`, terlihat proses dengan status Z (zombie). Hal ini terjadi karena proses anak telah selesai dieksekusi menggunakan `exit(0)`, namun proses induk tidak memanggil fungsi `wait()` untuk mengambil status `exit` dari proses anak. Selama proses induk masih berjalan (sleep 60 detik), proses anak tetap berada dalam tabel proses sebagai zombie.

C. Tugas Pemrograman

1. Buatlah sebuah program untuk menghitung statistik data (Maksimum, Minimum, dan Rata-rata) dari argumen yang diberikan melalui *command line*. Namakan program sebagai `child_stat.c`. Kompilasi program untuk menghasilkan berkas tereksekusi (*executable file*): `stat_worker.out`. Proses ini akan anda gunakan sebagai proses anak.

Jawab :

Program `child_stat.c` dibuat untuk menghitung nilai maksimum, minimum, dan rata-rata dari data yang diberikan melalui *command line* argument. Program menerima beberapa angka sebagai parameter, kemudian memprosesnya dan menampilkan hasil statistik.



```
Kaleh@Witz: ~/TugasPemrograman
Kaleh@Witz:~/TugasPemrograman$ cat child_stat.c
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    int i;
    int num;
    int max, min;
    float sum = 0;

    if(argc < 2){
        printf("Masukkan angka sebagai argumen\n");
        return 1;
    }

    max = min = atoi(argv[1]);

    for(i = 1; i < argc; i++){
        num = atoi(argv[i]);

        if(num > max)
            max = num;

        if(num < min)
            min = num;

        sum += num;
    }

    printf("Maximum : %d\n", max);
    printf("Minimum : %d\n", min);
    printf("Average : %.2f\n", sum/(argc-1));

    return 0;
}
Kaleh@Witz:~/TugasPemrograman$ gcc child_stat.c -o stat_worker.out
Kaleh@Witz:~/TugasPemrograman$ ./stat_worker.out 10 20 30 40
Maximum : 40
Minimum : 10
Average : 25.00
Kaleh@Witz:~/TugasPemrograman$ S
```

Program `child_stat.c` kemudian dikompilasi menggunakan GCC dengan perintah `gcc child_stat.c -o stat_worker.out`. Setelah berhasil dikompilasi, program dijalankan dengan beberapa argumen angka untuk membuktikan bahwa program dapat menghitung maksimum, minimum, dan rata-rata dengan benar.

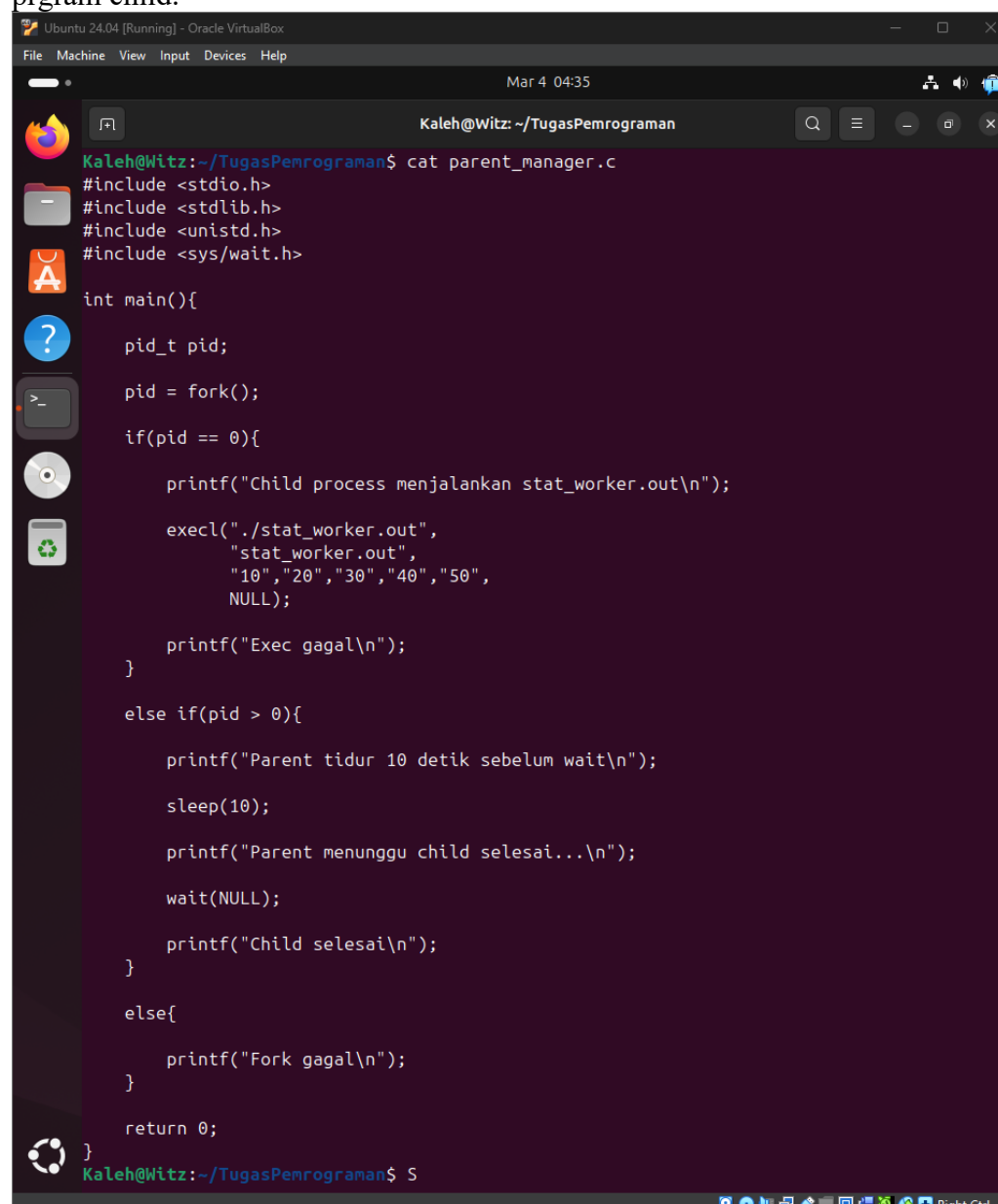
2. Buatlah proses induk (`parent_manager.c`) yang bertujuan untuk mengirimkan deretan angka kepada proses anak. Mahasiswa diberikan kebebasan dalam metode pemberian input, yaitu pilih salah satu:

- **Metode A (Interaktif):** Program induk meminta input jumlah data dan angka-angka dari pengguna.
- **Metode B (Statis):** Program induk sudah menentukan deretan angka di dalam kode.

Proses induk kemudian harus menciptakan proses anak menggunakan `fork()` dan menjalankan berkas `stat_worker.out` menggunakan fungsi `exec()`. Data angka wajib dikirimkan sebagai argumen ke proses anak melalui fungsi `exec` tersebut.

Jawab :

Program `parent_manager.c` berfungsi sebagai proses induk yang akan membuat proses anak menggunakan fungsi `fork()`. Proses anak kemudian menjalankan program `stat_worker.out` menggunakan fungsi `exec()`. Program ini juga mengirimkan deretan angka sebagai argumen ke program child.



```
Kaleh@Witz: ~/TugasPemrograman
Kaleh@Witz:~/TugasPemrograman$ cat parent_manager.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(){
    pid_t pid;

    pid = fork();

    if(pid == 0){
        printf("Child process menjalankan stat_worker.out\n");

        execl("./stat_worker.out",
              "stat_worker.out",
              "10", "20", "30", "40", "50",
              NULL);

        printf("Exec gagal\n");
    }

    else if(pid > 0){
        printf("Parent tidur 10 detik sebelum wait\n");

        sleep(10);

        printf("Parent menunggu child selesai...\n");

        wait(NULL);

        printf("Child selesai\n");
    }

    else{
        printf("Fork gagal\n");
    }

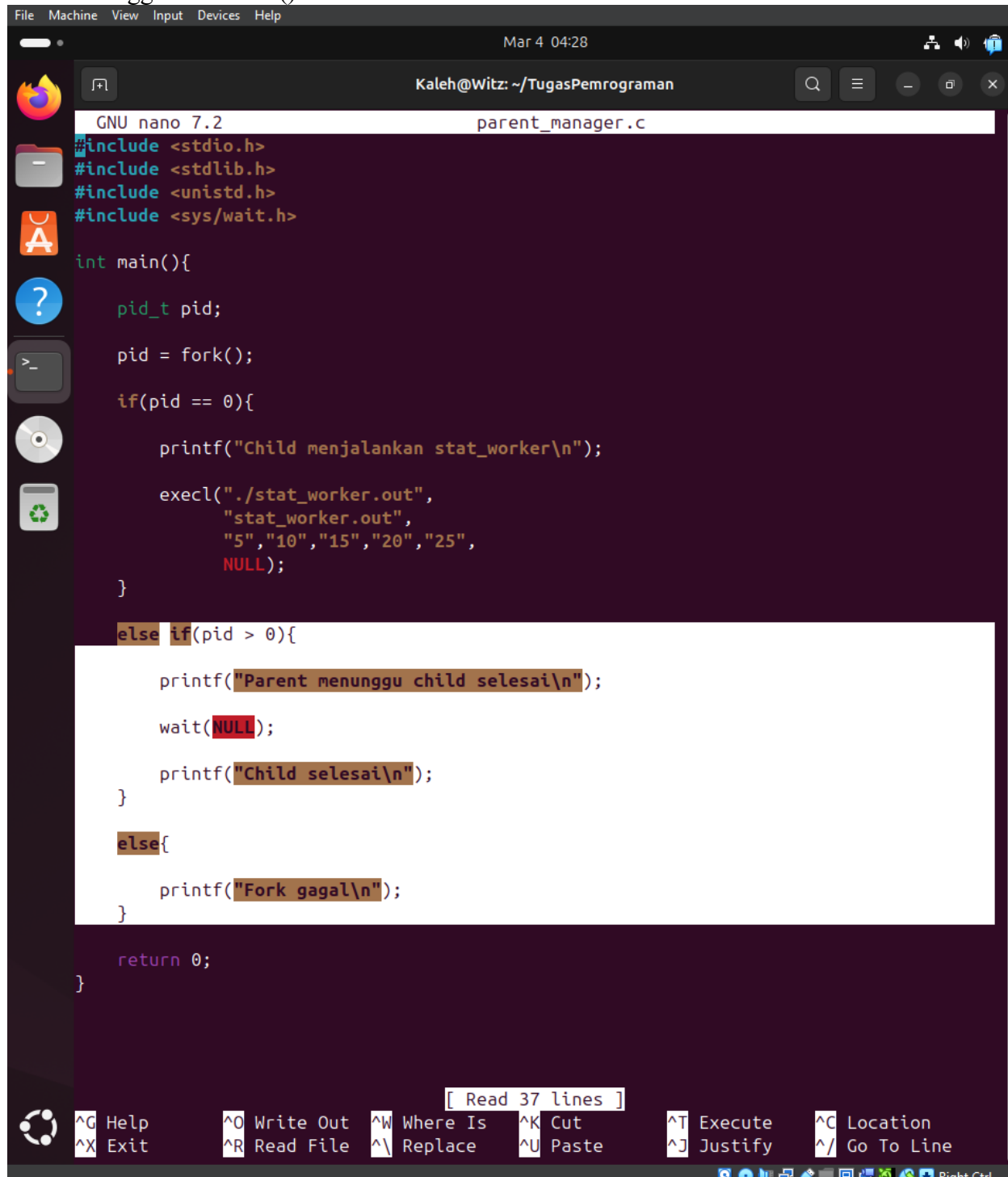
    return 0;
}
```

```
Ubuntu 24.04 [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Mar 4 04:23
Kaleh@Witz: ~/TugasPemrograman
Kaleh@Witz:~$ mkdir TugasPemrograman
Kaleh@Witz:~$ cd TugasPemrograman
Kaleh@Witz:~/TugasPemrograman$ ls
Kaleh@Witz:~/TugasPemrograman$ nano child_stat.c
Kaleh@Witz:~/TugasPemrograman$ gcc child_stat.c -o stat_worker.out
Kaleh@Witz:~/TugasPemrograman$ ls
child_stat.c  stat_worker.out
Kaleh@Witz:~/TugasPemrograman$ ./stat_worker.out 10 20 30 40 50
Maximum : 50
Minimum : 10
Average : 30.00
Kaleh@Witz:~/TugasPemrograman$ nano parent_manager.c
Kaleh@Witz:~/TugasPemrograman$ gcc parent_manager.c -o parent_manager.out
Kaleh@Witz:~/TugasPemrograman$ ./parent_manager.out
Parent menunggu child selesai...
Child process menjalankan stat_worker
Maximum : 50
Minimum : 10
Average : 30.00
Child selesai
Kaleh@Witz:~/TugasPemrograman$ ls
child_stat.c  parent_manager.c  parent_manager.out  stat_worker.out
Kaleh@Witz:~/TugasPemrograman$ S
```

3. Proses induk harus memastikan bahwa ia menunggu hingga proses anak selesai melakukan perhitungan dan menampilkan hasilnya ke layar sebelum proses induk mengakhiri eksekusinya sendiri. Gunakan fungsi `wait()` untuk sinkronisasi ini.

Jawab :

Program `parent_manager.c` dikompilasi menggunakan GCC dan dijalankan. Pada saat dijalankan, proses parent akan membuat child process. Child process akan mengeksekusi `stat_worker.out` dan menampilkan hasil perhitungan statistik, sementara parent menunggu child selesai menggunakan `wait()`.



The screenshot shows a terminal window with the nano text editor open, editing the file `parent_manager.c`. The code is as follows:

```
GNU nano 7.2 parent_manager.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(){
    pid_t pid;

    pid = fork();

    if(pid == 0){
        printf("Child menjalankan stat_worker\n");

        execl("./stat_worker.out",
              "stat_worker.out",
              "5", "10", "15", "20", "25",
              NULL);
    }

    else if(pid > 0){
        printf("Parent menunggu child selesai\n");
        wait(NULL);
        printf("Child selesai\n");
    }

    else{
        printf("Fork gagal\n");
    }

    return 0;
}
```

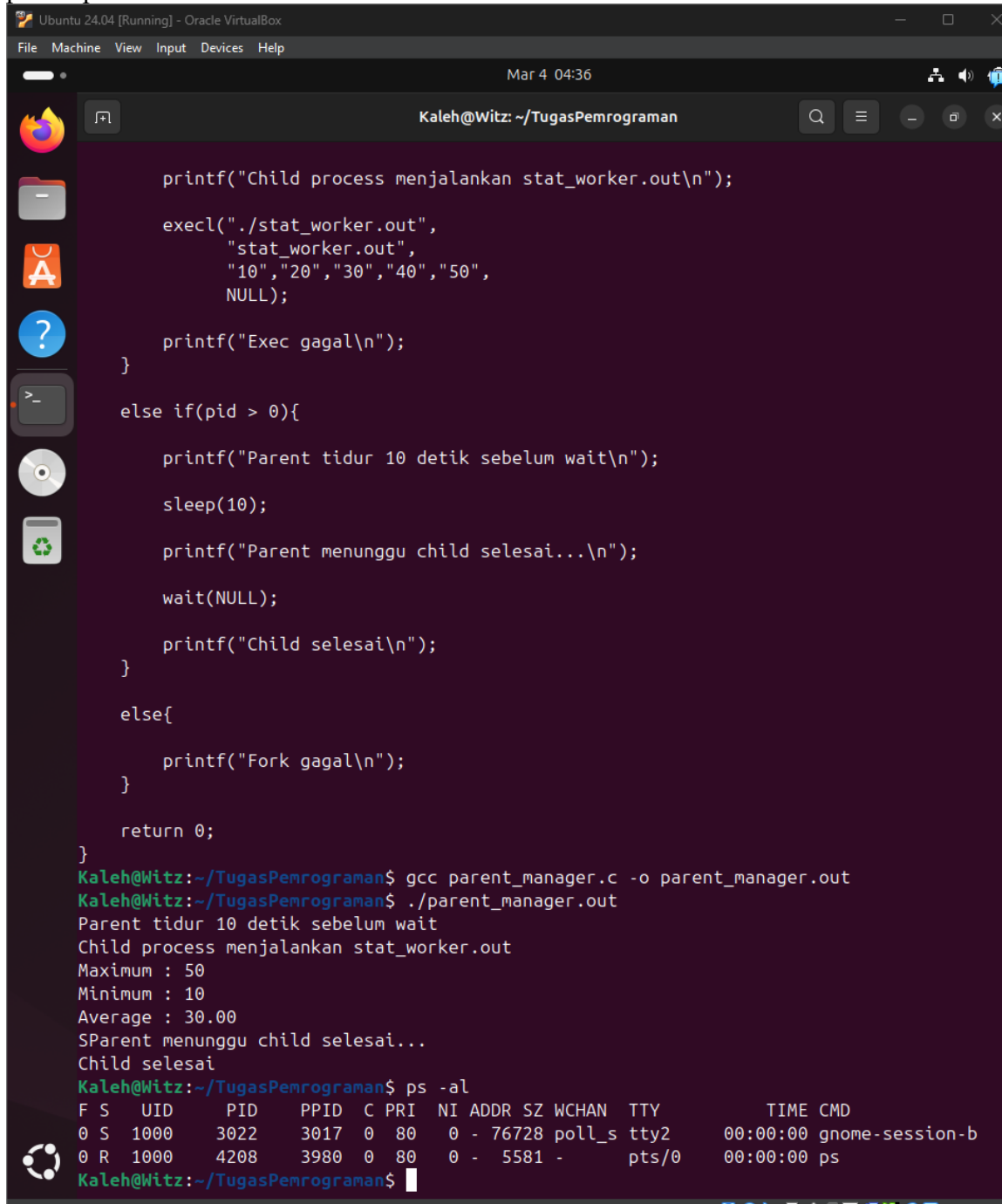
At the bottom of the terminal, a status bar shows "[Read 37 lines]" and a list of keyboard shortcuts: `^G Help`, `^X Exit`, `^O Write Out`, `^R Read File`, `^W Where Is`, `^_ Replace`, `^K Cut`, `^U Paste`, `^T Execute`, `^J Justify`, `^C Location`, and `^_ Go To Line`. The system clock at the top right indicates "Mar 4 04:28".

```
File Machine View Input Devices Help
Mar 4 04:29
Kaleh@Witz: ~/TugasPemrograman
Kaleh@Witz:~$ mkdir TugasPemrograman
Kaleh@Witz:~$ cd TugasPemrograman
Kaleh@Witz:~/TugasPemrograman$ ls
Kaleh@Witz:~/TugasPemrograman$ nano child_stat.c
Kaleh@Witz:~/TugasPemrograman$ gcc child_stat.c -o stat_worker.out
Kaleh@Witz:~/TugasPemrograman$ ls
child_stat.c  stat_worker.out
Kaleh@Witz:~/TugasPemrograman$ ./stat_worker.out 10 20 30 40 50
Maximum : 50
Minimum : 10
Average : 30.00
Kaleh@Witz:~/TugasPemrograman$ nano parent_manager.c
Kaleh@Witz:~/TugasPemrograman$ gcc parent_manager.c -o parent_manager.out
Kaleh@Witz:~/TugasPemrograman$ ./parent_manager.out
Parent menunggu child selesai...
Child process menjalankan stat_worker
Maximum : 50
Minimum : 10
Average : 30.00
Child selesai
Kaleh@Witz:~/TugasPemrograman$ ls
child_stat.c  parent_manager.c  parent_manager.out  stat_worker.out
Kaleh@Witz:~/TugasPemrograman$ nano parent_manager.c
Kaleh@Witz:~/TugasPemrograman$ gcc parent_manager.c -o parent_manager.out
Kaleh@Witz:~/TugasPemrograman$ ./parent_manager.out
Parent menunggu child selesai
Child menjalankan stat_worker
Maximum : 25
Minimum : 5
Average : 15.00
Child selesai
Kaleh@Witz:~/TugasPemrograman$ nano parent_manager.c
Kaleh@Witz:~/TugasPemrograman$ gcc parent_manager.c -o parent_manager.out
Kaleh@Witz:~/TugasPemrograman$ ./parent_manager.out
Parent menunggu child selesai
Child menjalankan stat_worker
Maximum : 25
Minimum : 5
Average : 15.00
Child selesai
Kaleh@Witz:~/TugasPemrograman$ S
```

4. Modifikasi program induk dengan menambahkan fungsi `sleep()` sebelum memanggil `wait()`. Lakukan observasi menggunakan perintah `ps -al` pada terminal yang berbeda saat program sedang berjalan. Analisis dan jelaskan status proses anak yang anda temukan pada kolom S, PID, dan PPID.

Jawab :

Pada tahap ini program dimodifikasi dengan menambahkan fungsi `sleep()` sebelum pemanggilan `wait()`. Hal ini bertujuan agar proses child masih dapat diamati menggunakan perintah `ps -al` pada terminal lain. Dari hasil pengamatan dapat dilihat hubungan antara PID (Process ID) dan PPID (Parent Process ID), yang menunjukkan bahwa child process merupakan turunan dari parent process.



```
printf("Child process menjalankan stat_worker.out\n");

execl("./stat_worker.out",
      "stat_worker.out",
      "10", "20", "30", "40", "50",
      NULL);

printf("Exec gagal\n");
}

else if(pid > 0){

    printf("Parent tidur 10 detik sebelum wait\n");

    sleep(10);

    printf("Parent menunggu child selesai...\n");

    wait(NULL);

    printf("Child selesai\n");

}

else{

    printf("Fork gagal\n");

}

return 0;
}
Kaleh@Witz:~/TugasPenrograman$ gcc parent_manager.c -o parent_manager.out
Kaleh@Witz:~/TugasPenrograman$ ./parent_manager.out
Parent tidur 10 detik sebelum wait
Child process menjalankan stat_worker.out
Maximum : 50
Minimum : 10
Average : 30.00
SParent menunggu child selesai...
Child selesai
Kaleh@Witz:~/TugasPenrograman$ ps -al
F S  UID      PID     PPID  C  PRI  NI ADDR SZ WCHAN  TTY      TIME  CMD
0 S  1000      3022     3017  0   80   0 - 76728 poll_s  tty2     00:00:00 gnome-session-b
0 R  1000      4208     3980  0   80   0 - 5581 -      pts/0     00:00:00 ps
Kaleh@Witz:~/TugasPenrograman$
```

EOF